# Best Practices for Machine Readable Data and Code Sharing in Psychology

Daniël Lakens[1], Cristian Mesquida[2], and Lisa DeBruine[3]

[1]Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

[2]Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

[3]Institute of Neuroscience and Psychology, University of Glasgow

## Author Note

Daniël Lakens https://orcid.org/0000-0002-8393-5316

Cristian Mesquida https://orcid.org/0000-0002-1542-8355

Lisa DeBruine https://orcid.org/0000-0002-7523-5539

A reproducible version of this manuscript available at on GitHub (https://github.com/scienceverse/machine_readable_FAIR_data_and_code). The authors have no conflict of interest to declare. This work was funded with the support of the Netherlands Organization for Scientific Research (NWO Vici grant VI.C.241.013). Author roles were classified using the Contributor Role Taxonomy (CRediT; https://credit.niso.org/) as follows: Daniël Lakens: conceptualization, writing, funding acquisition; Cristian Mesquida: conceptualization, writing; Lisa DeBruine: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Email: D.Lakens@tue.nl

**Abstract**

The FAIR principles state that research materials should be Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016). Best practices for FAIR data and code involve the creation of consistently structured repositories and standardised, machine-readable meta-data, but adhering to these best practices is a complex and effortful task. Previous literature has disussed how to adhere to these best practices with a focus on data and meta-data. Here we describe a variety of best practices and existing standards, with a practical focus on repository content and organization and introduce software that can facilitate these practices.

*Keywords:* FAIR, Data sharing, Reproducibility, Open Science

**Best Practices for Machine Readable Data and Code Sharing in Psychology**

Access to the data and code that underlie findings in the scientific literature allows peers to scrutinize the evidential basis for empirical claims, and to increase the efficiency of knowledge generation through re-use. The European Code of Conduct for Research Integrity states: "Researchers, research institutions, and organisations ensure that access to data is as open as possible, as closed as necessary, and where appropriate in line with the FAIR Principles (Findable, Accessible, Interoperable and Reusable) for data management." (ALLEA, 2023). Similarly, the Dutch Code of Conduct for Research Integrity states that: "Ensure that, in accordance with the FAIR principles, data is open and accessible to the extent possible and remains confidential to the extent necessary." (KNAW et al., 2018) Accordingly, research materials should be managed in accordance with the FAIR Principles (Wilkinson et al., 2016). Meeting these standards requires repositories to preserve the information needed to locate, understand, and use the deposited research outputs. To this purpose, Data management plans (DMPs) help researchers specify how data will be managed, documented, preserved, and shared throughout a project (Michener, 2015; Miksa et al., 2019). Our recommendations complement this planning by providing practical guidance on how to chose a repository and how to organize and document the different types of files within a research repository to support implementation of the FAIR principles.

Following a data management plan, making research files available and including an appropriate license does not, by itself, ensure that others will be able to use them. In psychology, Hardwicke et al. (2021) examined 25 articles awarded open-data badges between 2014 and 2016 and reproduced the target results. For 64% of the articles there were numerical discrepancies when trying to reproduce the results, after contacting the authors 37 out of 789 checked values remained unmatched. The principal obstacle was the unclear reporting of analytical procedures. Later, Trisovic et al. (2022) examined more than 9,000 R files from replication datasets deposited in Harvard Dataverse and 74% failed to compile without errors during initial execution. They also report how it is essential to use descriptive file naming, and found that about one third of the files (32%) contained a space in the name, a highly discouraged practice. These studies show how code and data sharing is not enough to

guarantee reuse. Such barriers prevent the ability to properly inspect materials, adapt code, or reuse the data for a new research question.

The FAIR Principles provide a framework for improving the Findability, Accessibility, Interoperability, and Reusability of research outputs by both people and machines (Wilkinson et al., 2016). Their emphasis extends beyond whether software can read a file: automated systems also need information about what a resource contains, how it relates to other resources, and under which conditions it can be used. Those Principles were explicitly developed with both human and machine users in mind. Although it is important that individual can find, access, and re-use data, the benefits of FAIR data and code can only be realised when scientific resources are also accessible and processable by machines. Therefore, research data and code should be shared in machine-readable formats (e.g., see Lakens & DeBruine, 2021 for an example of the benefits of machine-readable hypothesis). Machine-readable FAIR data and code will make it possible to retrieve and integrate data into large, searchable databases, and facilitate the automation of tasks, such as checking data and code for adherence to best research practices. Applying FAIR to research software introduces additional considerations, including software versions, dependencies, and the relationships among software components. These considerations motivated the development of the FAIR Principles for Research Software, or FAIR4RS (Barker et al., 2022; Chue Hong et al., 2022; Lamprecht et al., 2020). This guide summarises best practices for sharing data and code in ways that make them findable, accessible, interoperable, and reusable by both humans and machines.

A machine can only find data and code reliably and automatically if they have a stable digital identifier and metadata; can be accessed through standardized protocols (e.g., HTTPS or an Application Programming Interface (API)); are provided in formats that can be used by other software (e.g., non-proprietary formats accompanied by metadata describing how files should be read in); and are re-usable (e.g., by providing a codebook and a license).

When data and code are shared this way, an automated computational system can identify what files contain — including both their format and purpose — based on metadata or filenames; determine whether they can be accessed and reused, for example given the license or privacy regulations; and process the data and code accordingly. Ideally, each of these steps

should be unambiguous and deterministic, minimizing the need for probabilistic inference by machine-learning algorithms or large language models. This guide is intended for researchers and data managers. It discusses best research practices for making data and code machine-readable and FAIR, including where to share data and code, repository organization, file naming, file formats, codebooks, README and requirement files, data layout, column naming, codebook generation, specification of scales and measures and adherence to existing schemas and standards.

Practical guidance already addresses many aspects of sharing research outputs. Recommendations in neuroscience and biology explain how to prepare and document code for others (Abdill et al., 2024; Eglen et al., 2017), while minimum standards for data and code emphasise organised files, informative metadata, reusable formats, and explicit licences (Jenkins et al., 2023). More recently, TADA has proposed minimum requirements for sharing functional analytical code (Ivimey-Cook et al., 2025), and the SORTEE guidelines have established a staged framework for assessing archived data and code, from their availability and documentation to their correspondence with a manuscript and computational reproducibility (Pick et al., 2026).

The proposed word brings these recommendations together at the level of a complete research repository in psychology. A repository may contain data and code, codebooks, scripts, questionnaires, stimuli, generated outputs, and supporting documentation. Reusing these resources requires their purposes and relationships to be clear: which codebook describes a dataset, which script produces an output, which materials belong to a particular study, and which licence applies to each resource. Standardised organisation of files and explicit metadata can make these relationships easier to establish without requiring users to reconstruct them from the manuscript or contact the original authors.

This guide synthesises existing standards and practical recommendations into a resource organised around repository contents and file types. It covers repository choice, folder structure and naming, data formats and codebooks, code, outputs, materials, licences, and approaches to assessing FAIRness. The emphasis on machine-readable descriptions builds on broader efforts to make scientific information easier to discover and reuse automatically,

including machine-readable hypothesis tests (Lakens & DeBruine, 2021). The intended outcome is a repository from which people and software can identify the available resources, understand their relationships, retrieve relevant files, and determine the conditions for their reuse.

## 1 Repository Choice

There are many data repositories, each with different strengths and limitations. When choosing a data repository, some features should be taken into account: whether it allows you to share anonymized versions of the repository and specific files, or control who can access your data, whether the repository provides persistent identifiers (e.g., DOIs), stable versions, version control, appropriate metadata, and machine access. Other considerations include compatibility with regulations such as the General Data Protection Regulation (GDPR), funder requirements, and any limitations on the types or quantities of data that can be archived.

To support the choice of a repository, there are curated registries containing detailed information on data repositories such as re3data (re3data.org, n.d.) and OpenDOAR (Jisc, n.d.) allowing to filter repositories based on specific characteristics. In this section we will discuss a selection of commonly used repositories in psychology including Zenodo, GitHub, the Open Science Framework, ResearchBox, Figshare, and PsychArchives. Countries and universities may also provide dedicated research data repositories, such as the UK Data Service or DataverseNL. GitHub is primarily a platform for developing and collaborating on code rather than a data repository. Public GitHub releases can be archived through Zenodo, which assigns a DOI to each release and preserves a specific version. In Zenodo, files in a published version remain fixed; updates to files are represented as new versions, while the record also has a DOI that can point to all versions.

Table 1 provides a brief comparison of repositories commonly used in psychology and related social sciences. It is intended as an overview, not a ranking; institutional services based on platforms such as Dataverse or Figshare may differ from their general-purpose versions. Features and policies can change, so researchers should check the current terms and technical documentation for the specific service before depositing data. The comparison follows dimensions discussed in a recent overview of repository choice, including persistence,

machine access, metadata, peer-review sharing, usability, and data protection Lakens, 2026.

The suitability of a repository also depends on whether it supports machine access. Public APIs allow automated systems to retrieve metadata and files without relying on manual interaction with a web interface. ResearchBox now provides a documented API; PsychArchives has an API, although its documentation and programmatic access to some restricted files remain limitations. These features should be checked against the intended use of the shared materials.

Repositories that support anonymous or restricted-access sharing can be particularly useful during peer review. Anonymous links are available through PsychArchives, Dataverse, ResearchBox, and OSF; Zenodo and GitHub can also be used for review sharing, although this may take additional steps. Researchers should check what information a reviewer link exposes and whether the files themselves have been checked for identifying information.

For sharing anonymized data and research materials, Zenodo may be a suitable choice because it provides persistent DOIs, versioning, and options for restricting file access when necessary, including for materials under review or data that cannot be made publicly available. However, restricted access alone does not establish that a repository is appropriate for identifiable or sensitive personal data. Researchers should check applicable institutional, ethical, and data-protection requirements, particularly for pseudonymized data, before depositing it.

When choosing a data repository, therefore, do not simply ask whether it allows you to upload your files. Consider whether its infrastructure supports the level of openness, persistence, accessibility, and machine readability required for the research materials to adhere to FAIR principles.

## 2 Repository content and organization

Repositories should contain not only data but also materials that support *replication* and *reuse*. These may include code used to process data or run analyses, and study materials such as stimuli, questionnaires, and other experimental materials. These file types serve different purposes and may be reused in different ways. Study materials vary across projects and are not always machine-processable, whereas data and code often contain structured

information that people and software can access and process. This can support activities such as living meta-analyses or searches for studies that used a particular questionnaire. Repository metadata describe the project and its files, while codebooks explain variables and values in data files; both support interpretation and reuse.

### *2.1 Overall structure and file/folder naming conventions*

Naming conventions are not universal, so folder structures and names vary across projects and research fields. A few shared principles can nevertheless make repositories easier for people to navigate and for software to process.

**2.1.1 Folder structure.** Folders should group related files. A research repository can include folders named `data`, `code`, `output`, `materials`, and `documentation`, with an `other` folder for files that do not fit these categories. The top-level folder corresponds to the paper and contains files that apply to the repository as a whole. A README is one example: it describes the repository contents and gives instructions for reuse in a human-readable form.

Psych-DS, a data structure for social science research, recommends a subfolder for each project in which data were collected. For a multi-study paper, this means creating a folder for each study (e.g., `pilot-classification`, `study-mood-classification`, and `study-moderator-effect-mood`). Choose clear, consistent study labels early, for example when preparing an ethics application or preregistration. Because studies may be reported in a different order from the order in which they were conducted, and numbers alone do not describe a study, avoid naming folders only `study-1`, `study-2`, and so on. Short descriptive labels help connect repository files to the corresponding study in the manuscript. Once the order of studies in the paper is settled, numbers can be added to the folder names (e.g., `01-pilot-classification`, `02-study-mood-classification`).

At a minimum, each study folder should contain a `data` folder, and each data file should be accompanied by a codebook.

When data are analysed using individual code files, place them in a `code` folder. A study folder may also contain an `output` folder for results generated by analysis scripts. Repositories may include a `documentation` folder for files such as the preregistration and manuscript, and a `materials` folder for stimuli, survey files, and other study materials.

Software may search for these folders using a limited set of aliases. For example, a documentation folder might be called `reports` or `docs`, and a materials folder might be called `assets` or `stimuli`. If the range of names is limited, software can use a predefined list to identify folders containing particular types of files. This reduces reliance on large language models, which are less reliable and more computationally expensive than rule-based classification for this task.

```
repository/
   README.md
   code/
       data_cleaning.R
       analysis.R
       simulation.R
   data/
       raw_data.csv
       processed_data.csv
       codebook.csv
   output/
       results_simulation.csv
       table_results.csv
       figure_results.png
   materials/
       survey.txt
   documentation/
       preregistration.txt
   renv.lock
   LICENSE
```

**2.1.2 File and folder naming conventions.** Files and folders should be named so people can understand them and software can classify them. Because files may need to be identified by several properties (e.g., study identifier and file type), use a consistent naming pattern that makes these properties distinguishable. Underscores can separate properties: `study-01-classification_data.csv`, for example, identifies both the study (`study-01`) and the file type (`data`). For classification, using underscores or hyphens matters less than consistently including informative category terms. A CamelCase name such as `codePowerSimulationStudy1` can also be classified if it contains a recognizable category term.

Each filename should include a term that helps software assign it to a category.

Common categories are `data`, `code`, `output`, `documentation`, and `materials`. Include one of these terms, or a recognised synonym such as `data_dictionary` or `stimuli`, together with a descriptive keyword for the file's specific contents (e.g., `preprint_documentation.pdf`). Machines can classify files using filename and extension rules, with a large language model as a fallback for files that remain ambiguous.

Be consistent with capitalization because some software treats `Data.csv` and `data.csv` as different files. Lowercase names are generally preferable unless a schema or data standard requires capitalization. Avoid spaces and special characters. Use leading zeros so numbered files sort in the intended order (e.g., `02-data` before `11-data`). Choose enough digits for the largest expected sequence. Dates should use the ISO 8601 format `YYYYMMDD` (e.g., `20260724_data.csv`); the extended format `YYYY-MM-DD` may be used when required by a particular convention.

Long folder names can produce file paths that exceed limits imposed by file systems or software. Some systems limit filenames to 255 characters, and downloading a repository adds the local folder path to each file path. For example, if a repository is downloaded to `C:\Users\dlakens\Downloads`, the paths within the repository should remain below 228 characters. As a practical guide, keep directory paths below 100 characters and individual filenames below 50 characters, leaving room for the local path.

Names should match the terminology and identifiers used in the manuscript. If the paper refers to `Study 1`, for example, use the same study identifier in the corresponding folder rather than an unrelated label such as `Survey 1` or `Aggression Study`. This helps software link repository files to the relevant part of a multi-study paper. Metadata, including RO-Crate metadata discussed above, can describe file and folder contents that cannot be conveyed reliably through names alone.

**2.1.3 README and repository-level metadata.** Every repository should include a README. It helps people browsing the repository understand its contents and provides instructions for reuse. Because free-form README text is less reliable for automated processing, it should be complemented by structured metadata containing information that cannot easily be retrieved from the files themselves and is useful to re-users.

Structured metadata should describe the repository and its files, including information that is difficult to retrieve from the files themselves. It should identify the research project and its creators. Because machines may find a repository independently of the article, describe the project separately from the manuscript and include the manuscript DOI when one exists. Adding file descriptions to a JSON metadata file can also help check whether file names are unambiguous and provide a reliable description when they are not.

Repository metadata can be challenging because different standards operate at different levels. Dublin Core provides generic metadata about a repository, such as its title, description, creator, and license. The DDI Alliance provides ISO-standard specifications for describing research data in the social, demographic, economic, and health sciences. DDI-Lifecycle 3.3 can describe study design, sampling, and data to support reuse.

Field-specific guidelines are needed to clarify which metadata should be shared. Tools should make it easy to generate structured JSON metadata that follows widely used specifications. Such guidance could draw on existing standards, including DDI-Lifecycle 3.3 for study, sample, and data-handling descriptions; CRediT for contributor roles; Schema.org for bibliographic metadata; and the RO-Crate specification (Soiland-Reyes et al., 2022) for describing relationships among repository files and supporting machine-readable FAIR data and code.

Some information is difficult to retrieve from a paper but valuable for reuse, and can be recorded directly in metadata. Research fields should determine which information is important enough to include. In many empirical disciplines, sample size is useful for meta-analysis and helps indicate the strength of the evidence. DDI-Lifecycle provides standardized fields for sample eligibility, inclusion and exclusion criteria, recruitment, target sample size, completed responses, and response rate.

Metadata can also describe data provenance: how and under what circumstances the data were collected and handled. Useful details may include the collection period, whether sessions were supervised or conducted individually or in groups, whether collection took place during an exam period, and whether data were collected online or in person. DDI's `LifecycleEvent` schema allows researchers to record data-handling steps, including what

happened, when, and who was responsible. This can document contributions by research assistants, technicians, and data managers who are not authors, making their work visible and clarifying responsibility for each processing step. A schema for open science practices could also indicate whether shareable data, code, and materials have been made available and whether the study was preregistered.

Researchers may also share documentation such as codebooks, preregistration documents, the manuscript, supplementary materials, and README or metadata files. These documents provide context for interpreting and reusing data and code, so identify them clearly in the repository and will be discussed in detail in Section 2.2.4.

**2.1.4 Repository size, packaging, and selective downloads.** Online repositories often provide generous storage limits. For example, Zenodo has a standard limit of 100 files and 50 GB per record; additional storage can be allocated (file limits; storage quota). Large repositories can still make reuse difficult: users may lack the time or storage to download files they do not need, and automated tools may impose limits on the size of individual files or entire repositories. Organise and label files so users can download only the materials relevant to their purpose. For example, someone looking for analysis data should be able to retrieve the data and code without also downloading large video files used as stimuli.

Compression can reduce the size of large files. For multiple files, a `.zip` archive can be useful: its central directory lists the files it contains, including their names and sizes. Because this directory is near the end of the archive, software can retrieve it using HTTP range requests—when the server supports them—without downloading the entire archive. This allows users to check whether an archive contains relevant files before downloading it in full. Prefer ZIP to proprietary formats such as `.rar` when interoperability matters.

### *2.2 Data*

**2.2.1 Raw and processed data.** Where possible and appropriate, researchers should share raw data: the data in the form in which they were originally generated or collected. Examples include files exported from Qualtrics, audio recordings, and individual-participant EEG data. Raw data provide a record of the original observations and allow others to understand and reproduce the processing steps applied to them.

Sharing raw data is not always possible or appropriate. The data may not have been collected digitally, may contain information that cannot be adequately anonymised, or may be subject to ethical or legal restrictions. In such cases, share the earliest digital representation that can be made available, such as scanned surveys or anonymised transcripts. When data must be processed before they can be analysed or shared—for example, to remove identifying columns or convert unstructured data into a table—use code to perform these transformations whenever possible. Share the resulting minimally processed data together with the code used to create it. The README should explain how the processed data were created from the raw data and describe the transformations applied.

Using code rather than manually editing raw data has two advantages. It creates an auditable record of the transformations, allowing others to inspect the workflow for potential errors, and it makes the transformations reproducible: the same code can be rerun if errors are found, processing needs to be updated, or a new data version is required. Avoid manual changes to raw data; document any that cannot be avoided.

**2.2.2 File formats and data export.** Store data in open, machine-readable formats such as `.csv`, or in widely used software-specific formats such as `.sav`, `.jasp`, or `.omv`. For very large files, consider whether the format and repository support partial downloads through HTTP range requests. CSV files can allow parts of a file to be retrieved without downloading it in full, whereas formats such as `.xlsx` may require downloading the whole file.

When data are collected using commercial software, or on paper (e.g., a printed questionnaire), the raw data may not be available in an open format. In such cases, store the original files in a folder named `raw` inside the `data` folder, preferably in a `.zip` archive if there are many or large files. Also export the data, as directly as possible, to an open format such as CSV. CSV files should follow the RFC 4180 specification and use UTF-8 encoding. Enclose commas within data values in double quotation marks (e.g., `pp1, 34, "To be, or not to be."`). Avoid plain `.txt` files for tabular data unless their structure is documented, because the extension alone does not tell software how to interpret their contents. Similarly, use `.rds` files for individual R objects rather than storing them in `.RData` files, which can contain multiple objects and make their contents harder for software to identify. File extensions should

match the underlying format; for example, save a JSON file as `.json`, not `.csv` or `.txt`.

Compression can substantially reduce the size of large data files. Gzip (`.gz`) is particularly useful for large CSV files because compressed CSV files can be read directly without first being decompressed. We recommend gzip compression for CSV files larger than approximately 10 MB.

Spreadsheets (`.xls` or `.xlsx`) can also be used to store data. While the `.xlsx` format is based on Office Open XML and widely supported by open-source software, the `.xls` format relies on Microsoft's proprietary format and should be avoided to prioritize open-source alternatives. Do not store separate datasets on multiple tabs, because filenames cannot identify the contents of individual tabs; save each dataset as a separate file. Merged cells, empty rows or columns, and other layouts that depart from a rectangular table can also make spreadsheets difficult to import into data-analysis software.

Some analysis software, such as JASP and jamovi, can store data, codebook information, analyses, and output in a single file (`.jasp` or `.omv`). When all analyses can be conducted in one of these programs, sharing the resulting file may provide a simple way to make the data, labels, analyses, and output available together. This can require less setup than sharing separate data and code files, particularly for researchers who do not use a scripted workflow.

**2.2.3 Tabular data files.** Leave cells for missing values blank when the software used to read the data can interpret them as missing (e.g., as `NA` in R). Avoid ad hoc missing-value codes such as `999 = no response` or `888 = hardware failure`. If such codes must be retained—for example, because they are present in the raw data—document them in the metadata or codebook. Do not rely on colour, comments, or hyperlinks to convey information, as these may be lost when data are imported into other software; use dedicated columns instead (e.g., a `notes` column). CSV files and spreadsheets should have a single header row in the first row of the file.

Data files should be rectangular tables with a tidy structure: one header row, one variable per column, and one observation per row. Software such as SPSS, JASP, and jamovi expects this structure. Spreadsheet editing can introduce problems such as headers spanning

multiple rows, empty rows or columns, and notes placed below or alongside the data.

Column names should follow more restrictive conventions than filenames. Use only letters, digits, and underscores (e.g., `participant_id`, `mean_reaction_time`) to make names easier to reference in analysis code; in R, for example, a column can be accessed as `df$participant_id`. Names containing spaces or dashes must be quoted or accessed using another explicit syntax (e.g., `df[["participant id"]]`). Special characters, including diacritics, may need to be escaped or may be renamed when data are imported into other software, sometimes creating duplicate names. Keep names shorter than 32 characters, the limit in some analysis packages. Names must be unique within each file, consistent for the same variables across files, and identical to the corresponding names in the codebook.

Use descriptive column names that can be understood without consulting the analysis code (e.g., `self_esteem_scale_1` rather than `SE1`; avoid generic names such as `VAR1`). Where possible, use terminology that matches the manuscript. For items belonging to the same scale or construct, use a consistent prefix and sequential numbers (e.g., `self_esteem_scale_1`, `self_esteem_scale_2`, `self_esteem_scale_3`). This convention can also identify subscales (e.g., `panas_positive_1` for the first item of the Positive and Negative Affect Schedule's positive-affect subscale), making it easier for software to identify scales and subscales. Place items from the same scale in adjacent columns where possible. When several columns describe the same response, such as the response and its reaction time, make each column's meaning clear (e.g., `mood_1_response`, `mood_1_reaction_time`).

Avoid rows or columns containing only empty values, which may indicate that a value was not recorded, a column was deleted, or a row or column was accidentally left blank. Remove empty rows or columns where possible. If an empty variable must be retained, document why it is present rather than assigning an arbitrary value such as `0` or `none`. Column names and values should not contain leading or trailing whitespace (e.g., use `yes`, not `yes`). Avoid mixing text and numbers in otherwise numeric columns: a value such as `no response` can cause software to interpret the entire column as text. Use a consistent missing-value convention and document it in the codebook. Before sharing data publicly, check for personal information such as email addresses or IP addresses and remove or anonymise it as

appropriate. Screen open-text responses for identifying information as well.

**2.2.4 Metadata and codebooks.** Field-specific metadata standards can describe files commonly used in particular research areas, such as BIDS for brain data and Psych-DS for psychological and behavioral research data.

In addition to repository-level metadata, individual studies can be organized in Psych-DS-compliant subfolders. Each Psych-DS dataset requires a `dataset_description.json` file containing a machine-readable description of the dataset. Variable-level information, such as definitions and measurement details, can be provided in a data dictionary or related metadata. Unlike `ro-crate-metadata.json`, which describes provenance, contributions, and general relationships among files, these data descriptions help explain what the variables represent and support their reuse. (Psych-DS data dictionaries)

Machine-readable data descriptions support findability and reuse, and allow users to search for studies that collected particular types of data. Psych-DS uses Schema.org's Dataset model with additional Psych-DS fields. Structured descriptions can be generated automatically from well-organized repositories when the codebook is accurate and file and folder names are informative.

If a standalone codebook is needed, software packages can assist with its creation (Arslan, 2019).

A codebook, or data dictionary, describes the contents of data files; the terms are often used interchangeably. Buchanan et al. (2021) note that a codebook traditionally describes survey variables and may be created during data collection, whereas a data dictionary is often created during planning and may also describe the structure of data files. Regardless of the term used, each data file—or each set of files with the same structure—should have an accompanying codebook. Data files should have a single-row header, and the codebook should explain what the column names and values mean. Some formats (e.g., SPSS, Stata, SAS, JASP, jamovi, and Qualtrics) support variable and value labels, allowing some codebook information to be embedded in the data file.

A complete codebook should contain six types of information:

1. **Column name:** Use the name exactly as it appears in the data file (e.g., `panas_1`).

2. **Label:** Provide a plain-language description (e.g., `positive and negative affect schedule item 1`). For variables not derived from survey questions, describe what the values represent, such as reaction times or correct/incorrect responses. If a variable belongs to an established scale, include the scale's full name. At a minimum, provide the column name and label.
3. **Value labels:** For variables with a limited number of response options, explain what each value means (e.g., `1 = very negative; 2 = somewhat negative; 3 = neutral; 4 = somewhat positive; 5 = very positive`). If only the endpoints are labelled, document those (e.g., `1 = very negative; 5 = very positive`).
4. **Question or item text:** For survey data, include the verbatim question or item shown to participants.
5. **Missing-value codes:** If missing values are coded differently across columns, specify the code used for each variable.
6. **Coding instructions:** Describe any processing needed to interpret or calculate scores, such as reverse-scoring an item or calculating a mean from PANAS items 1–10. If instructions are too complex for a table, explain them in a README or provide code that transforms the data.

One way to create a codebook is to enter the relevant information in data-analysis software. In SPSS (`.sav`), Stata (`.dta`), SAS (`.sas7bdat`), JASP (`.jasp`), jamovi (`.omv`), and Qualtrics (`.qsf`), variable and value labels can be stored with the data. These formats can therefore combine data and some codebook information in one file, but may not capture every field needed for a complete codebook. For example, not all formats provide fields for documenting missing-value codes or the exact survey-item text, and none provides a dedicated field for coding instructions. Provide a separate codebook when information is missing from the data file.

Psych-DS provides a schema for describing data files and variables in JSON. Codebooks can also be tabular, with a column for each type of information recorded about a variable. An example is provided in Table 2.

Tabular codebooks can be stored as `.csv` files, but a spreadsheet may be easier to

create. Copy the column names from the data file and transpose them (i.e., convert the header row into a column) in the codebook so that names match exactly. Store values as text where needed, because Excel may convert values such as `1-5` into dates such as `5-Jan`.

Store codebooks as `.ods`, `.xlsx`, or `.csv` files. The first two formats are generally easier for people to read, while CSV is more interoperable. Avoid Word or PDF files because their contents cannot be reliably parsed by machines. Name a codebook `codebook` or `data-dictionary`. If a folder contains one data file, its location identifies the corresponding codebook; if it contains several, include the data filename in each codebook name (e.g., `pilot-1a_data.csv` and `pilot-1a_codebook.csv`).

### *2.3 Code*

Code files are part of the research record and should be organized so their purpose and relationship to other repository files are clear. These may include scripts, notebooks, and configuration files created or adapted for the project. Brief documentation should identify what each file does and which data or outputs it relates to, helping others understand and reuse the code.

The practice of clearly labeling files in repositories is also important to distinguish links to repositories that contain existing software used in research (e.g., a GitHub link to a software tool created by others that was used to process physiological data) from links to repositories that contain code generated by the researchers themselves for their research (e.g., code for an agent-based model that researchers have created as part of their research).

Computational reproducibility refers to the ability to produce the same outputs from the same data and code inputs (Lakens, n.d.). Computational reproducibility is often threatened by different versions of software, packages or computational settings. To increase computational reproducibility, the software and dependencies required to run the analysis should be clearly specified. For example, R projects can include an renv.lock file and Python projects a requirements.txt or environment.yml file. For analyses that depend on system libraries or software beyond R or Python packages, containerized solutions such as Docker, Nix, or Code Ocean provide stronger guarantees that the computational environment can be reconstructed in the future.

### *2.4 Output*

Researchers may also share output files such as statistical results, tables, figures, and simulation results. These files let users inspect research outputs directly. Outputs that can be regenerated from other repository files do not always need to be stored, but sharing them can be useful when they require substantial computation or help users verify reported results.

If an output is shared, place it in the `output` folder and use an informative filename (e.g., `results_simulation.csv`). This helps users identify and download the files they need. Because long-term storage uses space, retain outputs when they substantially reduce the computational burden of generating them or help verify reported results.

### *2.5 Materials*

Study materials may include questionnaires, rating scales, stimuli, survey files, and other resources used to conduct a study. Field-specific metadata standards can make structured materials, especially scales and questionnaires, easier to identify and reuse. The Open Scales Database, for example, provides a specification for scales in psychology.

The Open Scale Definition (OSD) is a machine-readable format for describing questionnaires and rating instruments. It is part of the Psychology Experiment Building Language, which was created for building and running psychological experiments (Mueller & Piper, 2014). An `.osd` file has two top-level parts: `definition`, which describes the scale's structure in a language-neutral way, and `translations`, which stores the scale text by language code so that the same structure can represent different language versions. The OSD specification includes `scale_info`, `likert_options`, `dimensions`, `items`, and `scoring`; an entry can be set to `NULL`. The `scale_info` field includes the scale's name, code, abbreviation, original publication citation, license, and other descriptive information. The remaining fields describe response options, dimensions and subscales, scoring, and item text in available translations. Machine-readable scale metadata can support reuse and promote consistency in how scales are used and scored across studies.

### *2.6 Licenses*

Sharing data and materials is not enough. Each repository should state clearly how its contents may be reused. It is important to check the default license of the chosen repository if

a specific license is not selected by the owner of the repository. A single license may not be appropriate for every file type: code generally requires a software-specific license (e.g., MIT or GPL), whereas data and other non-software outputs may be shared under a suitable Creative Commons license or public-domain dedication (e.g., CC BY or CC0). Creative Commons recommends against using its licenses for software because they do not address software-specific issues. Before applying a license, check that you have the rights or permission to license the material, and consider privacy, consent, funder or institutional requirements, and third-party rights. (Creative Commons FAQ; Choose a License)

Include the license text in a clearly named `LICENSE` file at the repository root. GitHub allows to include a license in the repository and automatically creates the file and file text based on the chosen license. If different parts of the repository have different licenses, use separate license files or state clearly in the README and repository metadata which license applies to which content. Check the repository's default license before depositing files: Zenodo, for example, defaults to CC BY 4.0 for a record and allows mixed licenses to be declared, such as MIT for software and CC BY for documentation. (Zenodo: Licenses and rights; GitHub: Licensing a repository)

## 3 How to check for FAIRness of a Repository

One solution to overcome the challenges of implementing all these recommendations and training of researcher is to automate the generation and validation of FAIR data and code as much as possible. Software could scan a repository and assist researchers in iteratively improving its FAIRness. Metacheck (https://github.com/scienceverse/metacheck) is one implementation of this approach: it scans a repository, classifies files, retrieves information from data files and codebooks, and can assist in creating metadata files. Input from researchers remains essential for information that cannot be inferred automatically, but manual inputs should only be required where needed.

## Discussion

Data and code should be shared where possible to facilitate verification, reproducibility and reuse. Repositories containing FAIR data and code should allow machines to find and access all content. This requires data and code to be made available in online repositories that

can feasibly be downloaded; files and folders to be clearly named and organized in a consistent folder structure to allow automatic classification and identification of relationships between files; the repository contents to be described using machine-readable metadata; files to be shared in open and machine-readable formats; a codebook to be included included; data files to be well-structured; and, where relevant, metadata about measures and tasks to be provided. Schemas such as Psych-DS, RO-Crate JSON-LD, DDI-Lifecycle, CRediT, Open Scales Definition, and Behaverse Data Model (Defossez et al., 2020) can facilitate FAIR data and code.

Adhering to best practices for FAIR data and code is complex and effortful. This document contains more than 100 recommendations and it is unreasonable to expect researchers or data managers to remember and implement them all. It becomes essential to assist researchers in the FAIRification of their research outputs with automated tools, such as the previously described Metacheck (https://github.com/scienceverse/metacheck). However, checking that metadata follow a schema cannot establish that their contents are accurate or complete. Researchers must still verify, for example, that units are correct, processing steps are adequately described, and a codebook reflects the shared dataset. Guidance on scientific code review similarly emphasises the value of informed human examination alongside technical checks (Hillemann et al., 2025; Ivimey-Cook et al., 2023; Rokem, 2024).

The potential of machine-readable data and code extends beyond individual repositories. If data repositories can be scanned automatically, information about their contents could be processed automatically, and stored in a central database. Researchers could then search for data files containing particular scales or tasks while filtering by characteristics of the sample such as demographic information. This could greatly facilitate the reuse of research data. Machine-readable code and data could also make automated reproducibility checks possible increasing the quality of scientific manuscripts, without increasing the burden on peer reviewers. More broadly, machine-readable repositories could help researchers demonstrate adherence to research-integrity requirements to FAIR data and code when possible. They could also allow universities, funders, and other research organizations to automatically check whether shared research outputs meet relevant data-sharing requirements.

As adding metadata to repositories remains challenging, and the number of best practices in data sharing makes it difficult to remember and implement them all, automated tools will therefore play a big role in the adoption of FAIR principles to data and code. If data and code can be processed by machines, it becomes possible not only to validate the extent to which a repository follows FAIR principles, but also to guide researchers interactively towards improving its findability, accessibility, interoperability, and reusability.

**References**

Abdill, R. J., Talarico, E., & Grieneisen, L. (2024). A how-to guide for code sharing in biology. *PLOS Biology*, *22*(9), e3002815. https://doi.org/10.1371/journal.pbio.3002815

ALLEA. (2023). *The european code of conduct for research integrity: Revised edition 2023*. ALLEA. https://doi.org/10.26356/ECOC

Arslan, R. C. (2019). How to automatically document data with the codebook package to facilitate data reuse. *Advances in Methods and Practices in Psychological Science*, *2*(2), 169–187. https://doi.org/10.1177/2515245919838783

Barker, M., Chue Hong, N. P., Katz, D. S., Lamprecht, A.-L., Martinez-Ortiz, C., Psomopoulos, F., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., & Honeyman, T. (2022). Introducing the FAIR principles for research software. *Scientific Data*, *9*, 622. https://doi.org/10.1038/s41597-022-01710-x

Buchanan, E. M., Crain, S. E., Cunningham, A. L., Johnson, H. R., Stash, H., Papadatou-Pastou, M., Isager, P. M., Carlsson, R., & Aczel, B. (2021). Getting started creating data dictionaries: How to create a shareable data set. *Advances in Methods and Practices in Psychological Science*, *4*(1), 2515245920928007. https://doi.org/10.1177/2515245920928007

Chue Hong, N. P., Katz, D. S., Barker, M., Lamprecht, A.-L., Martinez, C., Psomopoulos, F. E., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., Honeyman, T., Struck, A., Lee, A., Loewe, A., Werkhoven, B. van, Jones, C., Garijo, D., Plomp, E., Genova, F., … RDA FAIR4RS WG. (2022). *FAIR principles for research software (FAIR4RS principles)* (Version 1.0). Research Data Alliance. https://doi.org/10.15497/RDA00068

Defossez, A., Ansarinia, M., Clocher, B., Schmück, E., Schrater, P., & Cardoso-Leite, P. (2020). *The structure of behavioral data* (arXiv:2012.12583). arXiv. https://doi.org/10.48550/arXiv.2012.12583

Eglen, S. J., Marwick, B., Halchenko, Y. O., Hanke, M., Sufi, S., Gleeson, P., Silver, R. A., Davison, A. P., Lanyon, L., Abrams, M., Wachtler, T., Willshaw, D. J., Pouzat, C., & Poline, J.-B. (2017). Toward standard practices for sharing computer code and

programs in neuroscience. *Nature Neuroscience*, *20*(6), 770–773. https://doi.org/10.1038/nn.4550

Hardwicke, T. E., Bohn, M., MacDonald, K., Hembacher, E., Nuijten, M. B., Peloquin, B. N., deMayo, B. E., Long, B., Yoon, E. J., & Frank, M. C. (2021). Analytic reproducibility in articles receiving open data badges at the journal Psychological Science: An observational study. *Royal Society Open Science*, *8*(1), 201494. https://doi.org/10.1098/rsos.201494

Hillemann, F., Burant, J. B., Culina, A., & Vriend, S. J. G. (2025). *Code review in practice: A checklist for computational reproducibility and collaborative research in ecology and evolution*. EcoEvoRxiv; EcoEvoRxiv. https://doi.org/10.32942/X26S6P

Ivimey-Cook, E. R., Culina, A., Dimri, S., Grainger, M., Kar, F., Lagisz, M., Moran, N., Nakagawa, S., Roche, D., Tattan, S., Sánchez-Tójar, A., Windecker, S., & Pick, J. L. (2025). *TADA! Simple guidelines to improve analytical code sharing for transparency and reproducibility*. EcoEvoRxiv; EcoEvoRxiv. https://doi.org/10.32942/X2D93K

Ivimey-Cook, E. R., Pick, J. L., Bairos-Novak, K. R., Culina, A., Gould, E., Grainger, M., Marshall, B. M., Moreau, D., Paquet, M., Royauté, R., Sánchez-Tójar, A., Silva, I., & Windecker, S. M. (2023). Implementing code review in the scientific workflow: Insights from ecology and evolutionary biology. *Journal of Evolutionary Biology*, *36*(10), 1347–1356. https://doi.org/10.1111/jeb.14230

Jenkins, G. B., Beckerman, A. P., Bellard, C., Benítez-López, A., Ellison, A. M., Foote, C. G., Hufton, A. L., Lashley, M. A., Lortie, C. J., Ma, Z., Moore, A. J., Narum, S. R., Nilsson, J., O'Boyle, B., Provete, D. B., Razgour, O., Rieseberg, L., Riginos, C., Santini, L., … Peres-Neto, P. R. (2023). Reproducibility in ecology and evolution: Minimum standards for data and code. *Ecology and Evolution*, *13*(5), e9961. https://doi.org/10.1002/ece3.9961

Jisc. (n.d.). *OpenDOAR: Directory of open access repositories*. Retrieved November 4, 2026, from https://opendoar.ac.uk/

KNAW, NFU, NWO, TO2-federatie, Vereniging Hogescholen, & VSNU. (2018). *Netherlands code of conduct for research integrity*. https://storage.knaw.nl/2023-07/

Netherlands-Code-of-Conduct-for-Research-Integrity_2018_UK-september2018.pdf

Lakens, D. (n.d.). Computational reproducibility. In *Improving your statistical inferences*. Retrieved November 2, 2026, from https://lakens.github.io/statistical_inferences/14-computationalreproducibility.html

Lakens, D., & DeBruine, L. M. (2021). Improving transparency, falsifiability, and rigor by making hypothesis tests machine-readable. *Advances in Methods and Practices in Psychological Science*, *4*(2), 2515245920970949. https://doi.org/10.1177/2515245920970949

Lamprecht, A.-L., Garcia, L., Kuzak, M., Martinez, C., Arcila, R., Martin Del Pico, E., Dominguez Del Angel, V., Sandt, S. van de, Ison, J., Martinez, P. A., McQuilton, P., Valencia, A., Harrow, J., Psomopoulos, F., Gelpi, J. Ll., Chue Hong, N., Goble, C., & Capella-Gutierrez, S. (2020). Towards FAIR principles for research software. *Data Science*, *3*(1), 37–59. https://doi.org/10.3233/DS-190026

Michener, W. K. (2015). Ten simple rules for creating a good data management plan. *PLoS Computational Biology*, *11*(10), e1004525.

Miksa, T., Simms, S., Mietchen, D., & Jones, S. (2019). Ten principles for machine-actionable data management plans. *PLoS Computational Biology*, *15*(3), e1006750.

Mueller, S. T., & Piper, B. J. (2014). The Psychology Experiment Building Language (PEBL) and PEBL test battery. *Journal of Neuroscience Methods*, *222*, 250–259. https://doi.org/10.1016/j.jneumeth.2013.10.024

Pick, J. L., Allen, B. J., Bachelot, B., Bairos-Novak, K. R., Brand, J. A., Class, B., Dallas, T., D'Amelio, P. B., Fenollosa, E., Fernández-Juricic, E., Gomes, D. G. E., Grainger, M. J., Guillemaud, T., John, C., Krasnow, R., Lagisz, M., Lequime, S., Maynard, D. S., Nakagawa, S., … Ivimey-Cook, E. R. (2026). The SORTEE guidelines for data and code quality control in ecology and evolutionary biology. *Peer Community Journal*, *6*, e20. https://doi.org/10.24072/pcjournal.687

re3data.org. (n.d.). *Registry of research data repositories*. https://doi.org/10.17616/R3D

Rokem, A. (2024). Ten simple rules for scientific code review. *PLOS Computational Biology*, *20*(9), e1012375. https://doi.org/10.1371/journal.pcbi.1012375

Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L. J., Coppens, F., Fernández, J. M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., Ó Carragáin, E., Portier, M., Trisovic, A., RO-Crate Community, Groth, P., & Goble, C. (2022). Packaging research artefacts with RO-Crate. *Data Science*, *5*(2), 97–138. https://doi.org/10.3233/DS-210053

Trisovic, A., Lau, M. K., Pasquier, T., & Crosas, M. (2022). A large-scale study on research code quality and execution. *Scientific Data*, *9*, 60. https://doi.org/10.1038/s41597-022-01143-6

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L. B. da, Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

**Table 1**
*Comparison of selected research repositories*

| Repository | Main advantages | Limitations and points to check |
|---|---|---|
| Zenodo | DOIs, versioning, API, and options to restrict access to files. | No folder hierarchy; metadata remain public when files are restricted; file updates require a new version. (versioning; file access) |
| GitHub | Code development, collaboration, version history, releases, and API. Public releases can receive a DOI through Zenodo. | Primarily for code collaboration, not long-term data archiving; Zenodo DOI integration requires a public repository. (citing GitHub content) |
| Open Science Framework (OSF) | Project collaboration and view-only links for peer review. | No new projects or components from November 16, 2026; existing projects become read-only on February 19, 2027. (transition guidance) |
| ResearchBox | Search within data and code, automatically generated codebooks, and a documented API. | Check that its preservation arrangements, access options, and file limits fit the project. (API) |
| Figshare | DOIs, versioning, API, and support for a range of research outputs. | Storage limits and features can differ between Figshare.com and institutional services. (overview) |
| PsychArchives | Psychology-focused repository; links related research outputs and offers anonymized reviewer links. | Aimed for psychology and related disciplines; datasets must be anonymized; standard file limit is 1 GB, with larger deposits possible by request. API documentation and automated access to restricted files should be checked. (repository information) |
| DataverseNL | Dataset DOIs, versioning, metadata, access management, and API-based upload. | Depositing data requires an account; institutional policies and available support may differ. (benefits; access) |
| UK Data Service ReShare | Social-science repository with review of deposits and open or safeguarded access options. | Deposits must fit the UK Data Service's scope and policies; safeguarded files may require registration or permission. (repository information) |

**Table 2**
*Example of a codebook*

| variable-name | label | values | missing-values | question | coding-instructions |
|---|---|---|---|---|---|
| `pid` | Participant identifier | | | | |
| `age` | Age in years | | -99 | What is your age in years? | |
| `gender` | Self-reported gender | 1 = Man; 2 = Woman; 3 = Non-binary | 99 | Which best describes your gender? | |
| `bds_1` | Breakup Distress item 1 | 1 (Strongly disagree) to 5 (Strongly agree) | 99 | I feel overwhelmed by the breakup. | |
| `bds_3` | Breakup Distress item 3 | 1 (Strongly disagree) to 5 (Strongly agree) | 99 | I am coping well. | reverse scored: 6 - bds_3 |
| `bds_mean` | Breakup Distress mean score | | | | mean of bds_1 to bds_10 after reversing bds_3 |